From Structural Stability to Structural Transfer in Large Language Model Reasoning

Pre-experiment research draft

Abstract

Large language models may solve a reasoning problem in one presentation yet fail when the same dependencies are organized differently. A separate capability is transferring familiar operations to an unseen dependency graph. We distinguish these abilities as structural stability and structural transfer. Stability is measured as answer preservation on solver-validated, same-graph pairs; transfer is measured on a disjoint pool of unseen graph families and operator compositions. We further propose a falsifiable dependency-burden law based on relevant-premise distance and active dependency frontier size, and test it on a protected confirmatory split and an independent task family. StructShift is used as a measurement instrument, while LoRA is an optional boundary intervention. This is an experiment-before-results draft and makes no numerical claim.

1 Introduction

High accuracy on a conventional reasoning split does not answer two different questions. First, can a model preserve a solution that it has already demonstrated when the same dependency graph is presented in another legal form? Second, can it combine familiar operations into a dependency configuration that was absent from training? We call the first property *structural stability* and the second *structural transfer*. Conflating them into one out-of-distribution score makes it impossible to tell whether a failure reflects insufficient competence, presentation sensitivity, or a failure of systematic composition.

Compositional generalization studies show that models may learn task parts without reliably transferring to unseen structures [1–4]. Mathematical reasoning and context-interference evaluations show that controlled instantiations, distracting clauses, and information position can also change model behavior [5–8]. These settings manipulate different objects. A perturbation may alter a task graph, surface form, token length, context position, semantic solution, or rationale. A meaningful account therefore needs separate estimands and protected data partitions.

We propose a four-stage study. We first measure stability on answer-preserving same-graph pairs. We then measure transfer on disjoint held-out graph families and operator compositions. Next, we use solver traces to construct a dependency-burden error law based on relevant-premise distance and the active dependency frontier. We freeze its specification on a discovery split and test its predictions on a confirmatory split. Finally, we test its boundary on a second task family and under matched training interventions.

Our planned contributions are: (1) a non-circular distinction and measurement protocol for structural stability and structural transfer; (2) a solver-validated data construction with disjoint symbolic partitions; (3) a falsifiable, sample-out-of-sample dependency-burden law; and (4) boundary evidence that separates transferable robustness from local adaptation. No empirical result is claimed in this draft.

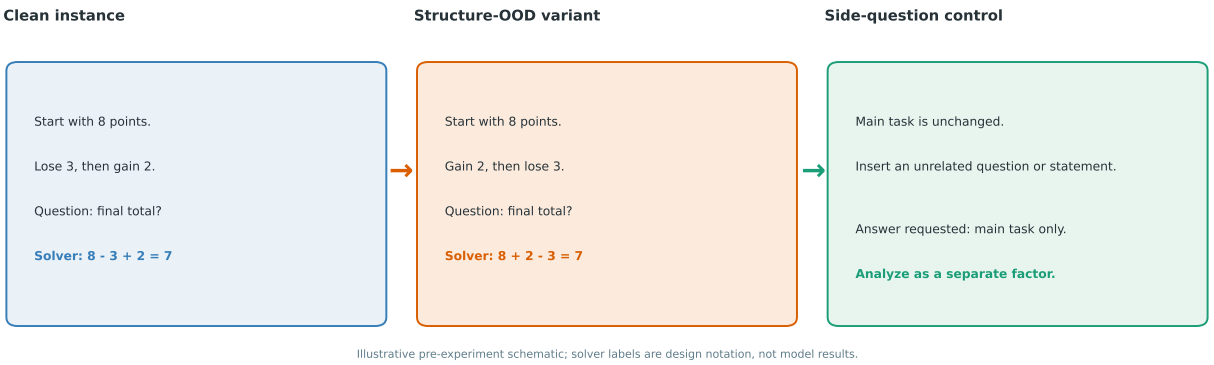


Figure 1: A solver-validated same-graph presentation change. The final figure will distinguish this stability pair from an independent unseen-graph transfer item.

2 Related Work

Compositional generalization. Compositionality work studies systematic transfer to unseen combinations [1–4]. We adopt structural holdouts for final-answer reasoning while keeping same-graph presentation stability as a separate estimand.

Reasoning robustness and context interference. GSM-Symbolic and GSM-Plus vary mathematical instances and clauses [5,6], while other work studies irrelevant context and information position [7,8]. We treat decoys and auxiliary questions as an interference pool rather than as direct evidence of unseen dependency composition.

Rationales and adaptation. Noisy-rationale and faithfulness studies manipulate intermediate explanations [9,10]. LoRA and QLoRA provide parameter-efficient adaptation [13,14], and targeted adversarial training prioritizes current model errors [15]. We use these ideas only for a matched boundary intervention; the scientific object is the relationship between stability, transfer, and observable dependency burden.

3 Stability and Transfer Framework

3.1 Problem formulation

Represent a latent reasoning task as $z = (G, q, y)$, let r denote its textual realization, and let the model input be

$$x = R(z, r). \tag{1}$$

Here G is a dependency graph, q is the target query, and y is the solver-verified answer. This notation separates the latent task from its surface realization.

3.2 Structural stability

A stability transformation T_s changes presentation while preserving the graph, query, and answer:

$$G(T_s x) = G(x), \qquad q(T_s x) = q(x), \qquad y(T_s x) = y(x). \tag{2}$$

Transformed accuracy mixes competence and stability. We therefore define the conditional stability rate and flip rate as

$$SS(f) = \Pr[f(T_s x) = y \mid f(x) = y], \qquad SF(f) = 1 - SS(f). \tag{3}$$

The condition $f(x) = y$ asks whether a demonstrated solution survives an equivalent presentation. As a check against conditioning artifacts, we also report agreement between two independently sampled equivalent realizations:

$$INV(f) = \mathbb{E}_{z,r,r'}\left[\mathbf{1}\{f(R(z,r)) = f(R(z,r'))\}\right]. \tag{4}$$

Agreement can be high when a model consistently makes the same mistake, so it does not replace SS.

3.3 Structural transfer

The transfer pool contains graph families or operator compositions excluded from training, discovery, and stability pools. We compare risks over seen and unseen graph distributions under the same canonical realization:

$$R_S(f) = \mathbb{E}_{G\sim P_S, q\sim P(q|G)}\left[\mathbf{1}\{f(x_G^{\text{can}}) \neq y_G\}\right], \tag{5}$$

$$R_U(f) = \mathbb{E}_{G\sim P_U, q\sim P(q|G)}\left[\mathbf{1}\{f(x_G^{\text{can}}) \neq y_G\}\right]. \tag{6}$$

The canonical-presentation transfer gap is

$$\Delta_T(f) = \Pr_{G\sim P_{\text{seen}}}[f(x_G^{\text{can}}) = y_G] - \Pr_{G\sim P_{\text{unseen}}}[f(x_G^{\text{can}}) = y_G]. \tag{7}$$

The two graph distributions are matched on primitive operators, length, labels, and templates. Unlike SS, Δ_T compares different graph supports under the same presentation condition. Equivalently, $\Delta_T = R_U - R_S$ is the increase in risk caused by changing the graph support. We also report normalized transfer retention

$$TR(f) = 1 - \frac{R_U(f) - R_S(f)}{1 - R_S(f)}, \qquad R_S < 1. \tag{8}$$

Stability conditions on a fixed z and varies r, whereas transfer fixes the canonical realization and compares two distributions over G; there is no identity linking the two quantities.

3.4 A two-by-two decomposition

Let $A_{g,p}$ denote accuracy for graph familiarity $g \in \{\text{seen}, \text{unseen}\}$ and presentation $p \in \{\text{canonical}, \text{shifted}\}$. We report

$$\Delta_S = A_{\text{seen,canonical}} - A_{\text{seen,shifted}}, \tag{9}$$

$$\Delta_T = A_{\text{seen,canonical}} - A_{\text{unseen,canonical}}, \tag{10}$$

and

$$I_{S\times T} = (A_{\text{seen,canonical}} - A_{\text{seen,shifted}}) - (A_{\text{unseen,canonical}} - A_{\text{unseen,shifted}}). \tag{11}$$

These are stability, transfer, and interaction contrasts, respectively; they are not relabeled versions of one score.

3.5 Finite theorems for a closure solver

The statements below are proved in Lean 4. Proofs are in Appendix A. Each statement constrains a closure solver and finite counterexamples. It is not a measurement of a trained model. Atoms are natural numbers. A rule is a premise set implying a conclusion. The solver answer is y. Order, synonym, premise deletion, and graph change are O_σ, $L^{\equiv}$, $L^{\times}$, and H_ψ. Extraction is φ. A closure solver is

$$f(x) = \mathbf{1}\{q \in \text{Closure}(\varphi(x))\}, \tag{12}$$

with $\varphi(T_s x) = \varphi(x)$ for every stability transformation T_s. When the fuel is at least the number of rules, closure membership coincides with definite-clause derivation.

Lemma 1 (Order and synonym)**.**

$$y(O_\sigma \omega) = y(\omega), \tag{13}$$
$$y(L^{\equiv}\omega) = y(\omega). \tag{14}$$

Thus O_σ and $L^{\equiv}$ are stability transformations. For the task ω_0 with an empty fact set, rule $\{0\} \to 1$, and query 1,

$$y(\omega_0) = \text{no}, \qquad y(L^{\times}\omega_0) = \text{yes}, \tag{15}$$

so $L^{\times}$ is not a stability transformation. H_ψ replaces $\{0\} \to 1$ by $\{0\} \to 2$. The two rule sets differ, so H_ψ is not a stability transformation either.

Lemma 2 (Accuracy gap and flip counts)**.** *Let each observation take $C, S \in \{0, 1\}$. On every finite list,*

$$\#\{C = 1\} - \#\{S = 1\} = \#\{C = 1, S = 0\} - \#\{C = 0, S = 1\}. \tag{16}$$

Write $\Delta = \#\{C = 1\} - \#\{S = 1\}$, $N = \#\{C = 1\}$, $F = \#\{C = 1, S = 0\}$, and $R = \#\{C = 0, S = 1\}$. Then

$$\Delta \cdot N = FN - RN, \qquad SF = \frac{F}{N} \quad (N > 0). \tag{17}$$

Hence $\Delta = 0$ does not imply $F = 0$. The counterexample is two lists of length 100, both with $N = 60$, and

$$SF \in \left\{0, \frac{30}{60}\right\}. \tag{18}$$

Lemma 3 (Seen-graph flips and the transfer gap)**.** *Two finite worlds $W_=$ and $W_{\neq}$, each of length 2, satisfy*

$$\#_{\text{seen}}\{C = 1\} = \#_{\text{unseen}}\{C = 1\} = 2, \qquad \Delta_T(W_=) = \Delta_T(W_{\neq}) = 0, \tag{19}$$

while the seen-graph flip counts are

$$F(W_=) = 0, \qquad F(W_{\neq}) = 1. \tag{20}$$

Lemma 4 (Interaction and presentation differences)**.** *Over the integers,*

$$I_{S\times T} = (A_{\text{sc}} - A_{\text{ss}}) - (A_{\text{uc}} - A_{\text{us}}), \tag{21}$$
$$J = (A_{\text{ss}} - A_{\text{us}}) - (A_{\text{sc}} - A_{\text{uc}}), \tag{22}$$

and

$$I_{S\times T} = -J. \tag{23}$$

Lemma 5 (Inversions under full reversal)**.**

$$\text{inv}(\text{rev}(m)) = \frac{m(m-1)}{2}, \qquad \text{inv}(\text{id}) = 0. \tag{24}$$

Therefore

$$\text{inv}(\text{rev}(2)) = 1 \neq 3 = \text{inv}(\text{rev}(3)). \tag{25}$$

This is a combinatorial identity, not an experimental measurement.

Theorem 6 (Orbits of a closure solver, and one counterexample function)**.** *If $\varphi \circ O_\sigma = \varphi$ and $\varphi \circ L^{\equiv} = \varphi$, then on that orbit*

$$f(x) = y \implies f(T_s x) = y, \tag{26}$$

so $SS(f) = 1$. If an unseen graph is still extracted correctly, then

$$f(x_U^{\text{can}}) = y \qquad \textit{and} \qquad f(T_s x_U) = y. \tag{27}$$

For the negative branch, a seen writing $\mathrm{seen}(t)$*,* $t \in \mathbb{N}$*, has rule* $\{0\} \to 1$*, fact* $\{0\}$*, and query* 1*. The unseen canonical writing has rule* $\emptyset \to 2$*, an empty fact set, and query* 2*. The rule sets are disjoint, and the solver answers* yes *on both. There is a function*

$$g(\mathrm{seen}(t)) = \mathrm{yes}, \qquad g(x_U^{\mathrm{can}}) = \mathrm{no}. \tag{28}$$

Thus g *is stable and correct on every writing of the seen graph, and incorrect on that unseen item. Reorder training supported only on seen writings does not exclude* g*.*

4 Predictive Dependency-Burden Law

We test a task-level account in which some errors arise from dependency tracking rather than length alone. Let $D_{\mathrm{rel}}(x)$ be the textual distance among proof-relevant premises and let $F_{\mathrm{active}}(x)$ be the maximum number of unresolved dependencies that must remain active before combination. The candidate model is

$$\mathrm{logit}\,\Pr(e_{m,x} = 1) = \alpha_m + \beta_1 D_{\mathrm{rel}}(x) + \beta_2 F_{\mathrm{active}}(x) + \gamma^\top Z(x), \tag{29}$$

where $Z(x)$ controls length, graph size, target position, template, label, depth, and branching. We choose the features, specification, and coefficient directions on a discovery split and evaluate once on a protected confirmatory split.

On the two-by-two data, we additionally fit

$$\mathrm{logit}\,\Pr(e_{m,x} = 1) = \alpha_m + \eta_S I_{\mathrm{shift}}(x) + \eta_T I_{\mathrm{unseen}}(x) + \eta_{ST} I_{\mathrm{shift}}(x) I_{\mathrm{unseen}}(x) + \beta^\top X(x). \tag{30}$$

The three coefficients represent presentation, unseen-graph, and interaction effects. They keep stability and transfer as separate error increments rather than collapsing them into one score.

4.1 A concrete law and falsifiable predictions

We do not call a correlation between the three structural features and error a law. The primary confirmatory model retains the standardized feature vector:

$$\mathrm{logit}\,\Pr(e_{m,x} = 1) = \alpha_m + \beta_D \widetilde{D}_{\mathrm{rel}}(x) + \beta_F \widetilde{F}_{\mathrm{active}}(x) + \beta_K \widetilde{K}_{\mathrm{op}}(x) + \gamma^\top Z(x). \tag{31}$$

Only after freezing weights on discovery do we define a scalar burden summary

$$B(x) = w_D \widetilde{D}_{\mathrm{rel}}(x) + w_F \widetilde{F}_{\mathrm{active}}(x) + w_K \widetilde{K}_{\mathrm{op}}(x), \qquad w_D, w_F, w_K \geq 0, \tag{32}$$

and formulate the law as a conditional error risk:

$$\Pr(e_{m,x} = 1 \mid B(x), Z(x)) = \sigma\left(\alpha_m + \phi(B(x)) + \gamma^\top Z(x)\right), \tag{33}$$

where $Z(x)$ controls length, graph size, depth, branching, target position, and template. The feature coefficients and any summary weights are frozen on discovery; a fixed monotone spline is used only for shape checking.

The falsifiable predictions are that the feature directions are preserved, higher frozen B yields higher error risk after conditioning on Z, the burden model outperforms the strongest length/graph-complexity baseline, and binned error rates increase monotonically with burden. Direction reversal, loss of predictive advantage, nonmonotone bins, or equal effects from matched placebo hints restrict the claim to an exploratory trend.

The account is weakened if local subproblems fail at the same rate as full proofs, nuisance-only models predict equally well, coefficient directions reverse on confirmation, or dependency hints and matched placebo hints have the same effect. The law is an observable prediction model, not a direct claim about hidden neural states.

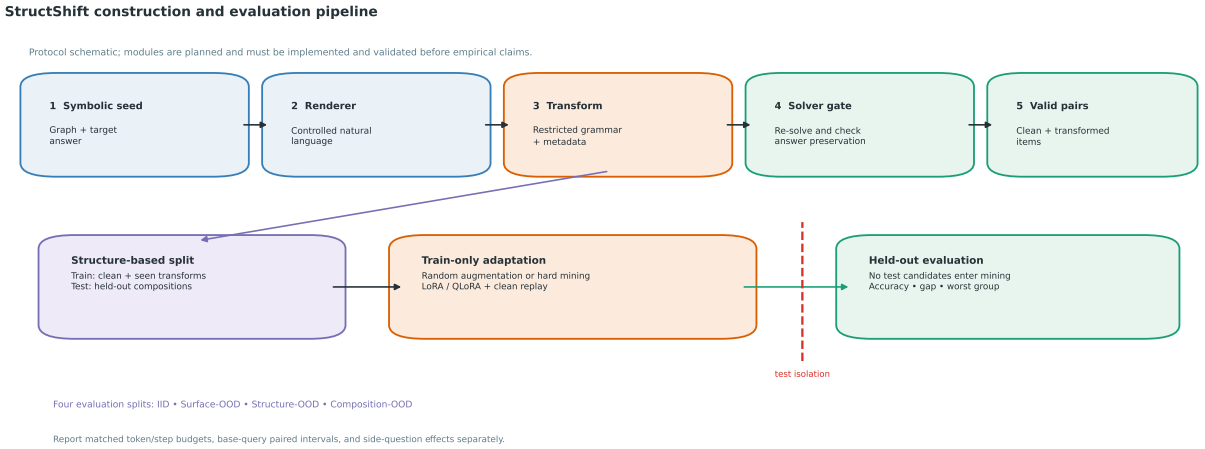


Figure 2: Solver-validated construction and protected symbolic partitioning. This is a pre-experiment schematic.

5 Measurement Instrument

StructShift creates three pools. The stability pool contains legal same-graph ordering and rendering changes. The transfer pool contains unseen graph families and operator compositions. Decoys and auxiliary questions form a separate interference pool. Symbolic objects are split before rendering, and each candidate is re-solved. Worlds, base-query IDs, graph hashes, graph-family IDs, proof traces, relevant-premise positions, active frontier size, token length, target position, template, and transformation lineage are retained.

6 Experimental Evidence Chain

6.1 E1: Structural stability

We evaluate canonical and same-graph variants with deterministic decoding. Increasing depth, branching, or changing operator composition is excluded from the stability definition. We report clean accuracy, transformed accuracy, conditional stability, conditional flips, recovery, and paired bootstrap intervals.

6.2 E2: Structural transfer

We evaluate held-out graph families and operator compositions under canonical presentation. Stability and transfer pools share no worlds, base queries, graph hashes, or graph families. Their relationship is tested only across multiple models, checkpoints, or controlled training conditions, while controlling independently measured clean competence.

6.3 E2.5: Orthogonal intervention and double dissociation

We compare four matched training conditions: Base, Render-Aug with only same-graph equivalent realizations, Graph-Aug with only canonical new graph compositions, and Joint with both. All conditions are evaluated on the same two-by-two cells and matched for training tokens, steps, LoRA rank, target modules, random seeds, and compute.

Define gains over Base as

$$G_S(c) = SS_c - SS_{\text{Base}}, \qquad G_T(c) = \Delta_{T,\text{Base}} - \Delta_{T,c}. \tag{34}$$

Render-Aug supports a stability-specific effect only when G_S exceeds a pre-registered margin while G_T falls inside the transfer equivalence margin; Graph-Aug uses the reverse criterion. Equivalence intervals are required instead of interpreting one significant and one nonsignificant result as dissociation.

6.4 E3: Out-of-sample error prediction

We compare length-only, graph-complexity-only, depth/branching, and full dependency-burden models. Confirmatory evaluation reports held-out log loss or Brier score, calibration, rank correlation, and error by burden bin. Local subquestions and dependency/placebo hints provide discriminating controls.

7 Applicability Boundary

Represent a test condition as

$$\tau = (\mathcal{G}, \mathcal{R}, \mathcal{Q}, m, d), \tag{35}$$

for graph distribution, realization distribution, task family, model, and decoding condition. After freezing the feature algorithms, normalization parameters, model specification, and coefficient directions on discovery, we define

$$V(\tau) = \big(V_{\text{direction}}(\tau), V_{\text{prediction}}(\tau), V_{\text{calibration}}(\tau)\big). \tag{36}$$

The applicability domain is

$$\mathcal{A} = \{\tau : V_{\text{direction}}(\tau) = V_{\text{prediction}}(\tau) = V_{\text{calibration}}(\tau) = 1\}. \tag{37}$$

Boundary experiments change one main factor at a time and record the task family, graph distribution, realization rule, model, intervention, and the three separate verdicts. This turns applicability into a measurable domain rather than a single second-task accuracy.

8 Boundary and Intervention

We repeat the frozen analysis on a second reasoning family with new templates and vocabulary. Only then do we compare a frozen base, clean-only LoRA, random legal structural augmentation, and optional error-guided structural LoRA under matched rank, target modules, tokens, steps, labels, seeds, and compute. Gains only on seen variants are local adaptation. Gains on protected transfer are intervention evidence, but do not alone prove a hidden neural mechanism.

8.1 Pre-experiment pipeline validation

Before connecting a language model, we ran a minimal validation of the data construction and metric pipeline. With random seed 20261008, we generated 25 symbolic graphs per family for two seen families (chain and fork_join) and two held-out families (chain_then_fork and double_chain_join), yielding 200 canonical/shifted records. Each shifted record changes only the presentation order of premises on the same dependency graph, and its solver answer must match the canonical answer. Held-out families are isolated at the symbolic level and do not enter the stability or discovery pool.

This check verifies the generator, answer-preservation checks for equivalent realizations, graph-family splitting, and metadata recording. Four unit tests cover pairing, answer preservation, and seen/unseen family isolation. These results establish only that the measurement pipeline runs as specified; they are not evidence about language-model robustness. Model accuracy, SS, SF, and Δ_T will be reported after real-model predictions are connected.

We also ran a non-confirmatory exploratory pilot with Qwen2.5-1.5B-Instruct on the same 200 records. Accuracy was 0.76 in all four cells, with $SS = 1.00$ and $\Delta_T = 0.00$. Each cell, however,

Table 1: Exploratory Qwen2.5-1.5B-Instruct pilot. Each cell contains 50 examples; label imbalance makes this a pipeline check rather than a confirmatory result.

Cell	n	Accuracy
Seen graph / canonical	50	0.76
Seen graph / shifted	50	0.76
Unseen graph / canonical	50	0.76
Unseen graph / shifted	50	0.76
Conditional stability SS	–	1.00
Transfer gap Δ_T	–	0.00

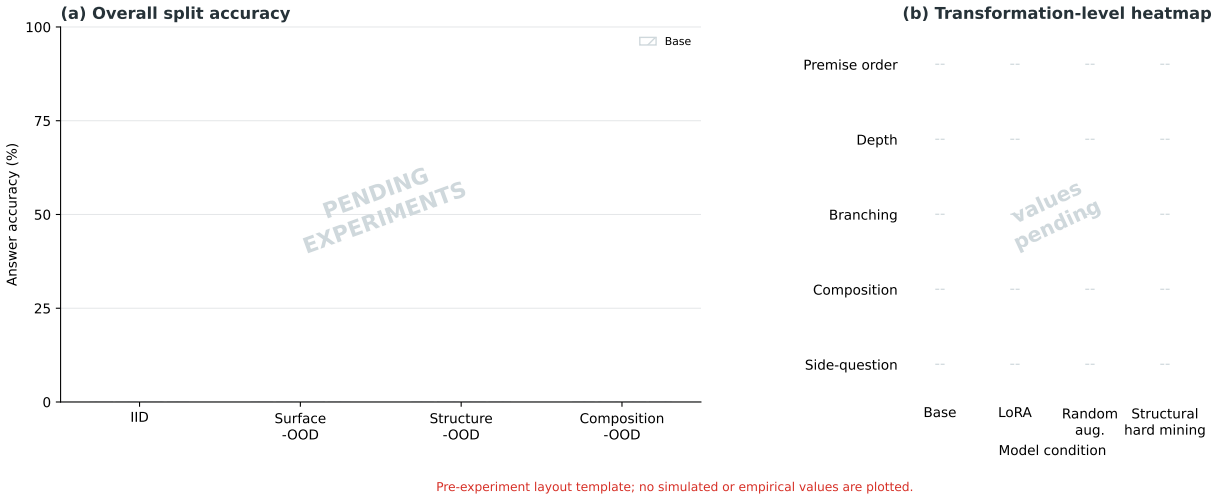


Figure 3: Planned result layout for stability, transfer, error prediction, and boundary tests. Values are filled after real-model experiments.

contains 38 "yes" and 12 "no" labels, and the model almost always outputs "yes". Thus 0.76 is the label baseline, while $SS = 1.00$ means only that the canonical and shifted outputs did not flip; neither result supports a structural-robustness claim. The confirmatory pilot must balance labels, randomize evaluation order, and include deeper, branched, and held-out compositions that produce measurable model errors.

9 Discussion and Limitations

The stability definition covers only presentation changes that preserve the same graph. Transfer depends on strict graph-family holdouts; weak holdouts can make surface changes look like structural transfer. The dependency law is a task-level predictive account and does not identify internal model states. Cross-model association between stability and transfer is not causal by itself. An intervention that improves only seen variants must be reported as local adaptation.

10 Conclusion

This paper separates structural stability from structural transfer in reasoning. The first is conditional answer preservation on the same graph; the second is performance on unseen dependency graphs under canonical presentation. A solver-validated measurement instrument, a frozen dependency-burden law, and independent-task and intervention tests together determine whether the observed pattern is stable, predictive, transferable, or task-specific.

A Proofs for the closure solver and finite counterexamples

The development is the Lake project `proof/StructRobust`, checked with Lean 4.35.0 and the core library only, without mathlib. The text below restates the arguments and does not paste tactic scripts. Each item is a theorem about a closure solver and finite counterexamples, not a measurement of a trained model.

A.1 Closure

File `proof/StructRobust/StructRobust/Closure.lean`, theorem `closure_fuel_iff_derives`. One round folds the rule list: if every premise is already known and the conclusion is not, the conclusion is added. Fuel $k + 1$ performs k rounds and then one more fold. Definite-clause derivation, `Derives`, has two clauses. A fact is derivable. If a rule belongs to the rule set and every premise is derivable, the conclusion is derivable.

Soundness is induction on fuel. A round adds the head of a rule whose premises are already known, so the new atom remains derivable from the original facts and rules. Completeness uses a potential: the number of rules whose heads are still missing. The potential is at most the number of rules. If a round does not reach a closed set, it adds at least one new head and the potential strictly decreases. Thus fuel at least the length of the rule list reaches a fixpoint, and every derivable atom has entered the closure. The solver accepts a query exactly when the query lies in that closure; see `solver_iff_derives` in `Operators.lean`.

A.2 Lemma 1

File `proof/StructRobust/StructRobust/Operators.lean`.

Order. Theorem `lemma1_order_preserves`: if two rule lists have the same members, `Derives` holds on one side exactly when it holds on the other, because a derivation uses only membership in the set (`derives_rules_congr` in `Closure.lean`). Theorem `lemma1_reorder_stable`: replacing the list while preserving membership leaves the fact set, the rule set, and the query unchanged, so the operator is a stability transformation `IsStability`. Theorem `stability_preserves` says every such transformation preserves the solver answer.

Synonym. A `Phrase` is a rule plus one wrapping field. Theorem `lemma1_synonym_denote`: changing the wrap does not change the denotation. Theorem `lemma1_synonym_extract`: extraction maps each phrase to its denotation, so wrapping every phrase leaves the extracted rule list unchanged.

Ambiguity. Theorem `lemma1_ambiguity_changes`. The task has no facts, one rule with premise 0 and head 1, and query 1. There is no fact to start from, and the only rule demands premise 0, so no atom is derivable. After the premise is deleted, the rule has an empty premise list and head 1, so 1 is derivable. The answers differ. Theorem `lemma1_ambiguity_not_stability`: a stability transformation would have preserved the answer.

Graph change. Theorem `lemma1_graph_not_stability`. One task has the rule $\{0\} \to 1$ and the other has $\{0\} \to 2$. The first rule is not a member of the second rule set, so the finite sets differ and the pair is not a stability transformation.

A.3 Lemma 2

File `proof/StructRobust/StructRobust/Counting.lean`. An observation is a pair of Booleans. Split $\#\{C = 1\}$ into the cells $S = 0$ and $S = 1$, and split $\#\{S = 1\}$ into the cells $C = 0$ and $C = 1$. Subtracting cancels the shared cell $\#\{C = 1, S = 1\}$ and yields theorem `lemma2_count_gap`. Theorem `lemma2_gap_times_correct` multiplies both sides by $\#\{C = 1\}$ and distributes. The subtracted term is $\#\{C = 0, S = 1\}$, the items that are wrong in the canonical writing and right after the transformation.

The two populations are concatenations of repeated observations. Theorem `lemma2_stable_counts`: sixty copies of $(C, S) = (1, 1)$ followed by forty copies of $(0, 0)$ have length 100, sixty canonical successes, sixty transformed successes, and flip count 0. Theorem `lemma2_flip_counts`: thirty copies each of $(1, 1)$, $(1, 0)$, and $(0, 1)$, plus ten copies of $(0, 0)$, also have length 100 and sixty canonical successes, of which thirty flip. Theorem `lemma2_same_accuracy_different_sf` records both populations together. Accuracy is 60/100 in each, while the flip counts are 0 and 30. Relative to the sixty canonically correct items, the conditional flip ratios are 0 and 30/60. The argument uses only finite counting.

A.4 Lemma 3

File `proof/StructRobust/StructRobust/Worlds.lean`, theorem `lemma3_same_transfer_different_sf`. In the first world the seen list is two copies of $(1, 1)$ and the unseen canonical marks are two truths. In the second world the seen list is one $(1, 0)$ and one $(1, 1)$, and the unseen marks are again two truths. Both worlds have two canonically correct seen items and two canonically correct unseen items, so both transfer gaps are 0. The seen-graph flip counts are 0 and 1, with denominator 2 in each world. The canonical transfer gap can therefore agree while the seen-graph flip count does not.

A.5 Lemma 4

File `proof/StructRobust/StructRobust/Interaction.lean`, theorem `lemma4_interaction_opposite`. Expand

$$I = (A_{\mathrm{sc}} - A_{\mathrm{ss}}) - (A_{\mathrm{uc}} - A_{\mathrm{us}})$$

and

$$J = (A_{\mathrm{ss}} - A_{\mathrm{us}}) - (A_{\mathrm{sc}} - A_{\mathrm{uc}})$$

over the integers. Then $I = A_{\mathrm{sc}} - A_{\mathrm{ss}} - A_{\mathrm{uc}} + A_{\mathrm{us}}$ and $J = -A_{\mathrm{sc}} + A_{\mathrm{ss}} + A_{\mathrm{uc}} - A_{\mathrm{us}}$. Their sum is 0, so $I = -J$.

A.6 Lemma 5

File `proof/StructRobust/StructRobust/Inversion.lean`. An inversion count adds, for each entry, the number of strictly smaller entries after it. The sorted list `canonOrder` is $0, 1, \ldots, m-1$. Every later entry is larger, so theorem `lemma5_canon_inversions` gives inversion count 0. The reversed list `revOrder` is $m-1, \ldots, 0$. In the inductive step, the list of length $m+1$ places m first. That entry is larger than each of the m entries after it, and the tail is the reversal of length m. The inversion counts therefore satisfy $a_{m+1} = a_m + m$ with $a_0 = 0$, hence $2a_m = m(m-1)$. Theorem `lemma5_reverse_inversions` divides by 2 and obtains $m(m-1)/2$. Theorems `lemma5_reverse_two` and `lemma5_reverse_three` evaluate the count at $m = 2$ and $m = 3$, giving 1 and 3. Theorem `lemma5_reverse_two_ne_three` records that these two doses differ. The count depends only on the permutation. It is not a measurement of model output.

A.7 Positive branch of Theorem 6

File `proof/StructRobust/StructRobust/Main.lean`. The function `fAnswer` extracts rules from the surface phrases and asks whether the query lies in the closure of that rule list. Theorem `main_positive_orbit`: if two surfaces extract the same set of rules, closure membership agrees, so f equals the solver answer on the canonical extraction. Theorem `main_positive_reorder`: if two phrase lists have the same members, their extracted rule sets agree, and reordering does not change f. Theorem `main_positive_synonym`: wrapping every phrase leaves the extracted list identical, so a synonym does not change f. Theorem `main_positive_finite_ss`: if f holds on the canonical writing and the transformed extraction is the same rule set, then f holds

after the transformation. Theorem `main_positive_finite_ss_all` applies this to every item of a finite list: every canonically correct item remains correct after the transformation. Theorem `main_positive_unseen`: if the canonical extraction of an unseen item already derives the query, then both the canonical writing and any writing with the same extracted rule set are correct.

A.8 Negative branch of Theorem 6

The same file. A writing of the seen graph carries a natural-number tag. Its facts are [0], its rule is $\{0\} \to 1$, and its query is 1. The fact 0 fires the rule, so 1 is derivable (`seen_derives`). The function `fArbitrary` returns true on every seen tag. Theorem `main_negative_seen_correct` says this matches the solver. Theorem `main_negative_seen_stable` says any two tags receive the same value, so every writing of the seen graph agrees. The unseen item has no facts, a rule with no premises and head 2, and query 2. The empty-premise rule derives 2 (`unseen_derives`), so the solver answer is true. `fArbitrary` returns false on that item. Theorem `main_negative_unseen_incorrect` says the function disagrees with the solver. Theorem `main_negative_disjoint`: the seen rule $\{0\} \to 1$ is not a member of the unseen rule set, so the two rule sets are disjoint and the unseen item is not a writing of the seen graph. Stability and correctness on the seen graph do not yield correctness on that unseen item.

References

[1] D. Hupkes, V. Dankers, M. Mul, and E. Bruni. Compositionality Decomposed: How Do Neural Networks Generalise? JAIR, 2020.

[2] J. Oren et al. Improving Compositional Generalization in Semantic Parsing. Findings of EMNLP, 2020.

[3] J. Herzig and J. Berant. Span-based Semantic Parsing for Compositional Generalization. ACL-IJCNLP, 2021.

[4] L. Qiu et al. Evaluating the Impact of Model Scale for Compositional Generalization in Semantic Parsing. EMNLP, 2022.

[5] I. Mirzadeh et al. GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models. arXiv, 2024.

[6] Q. Li et al. GSM-Plus: A Comprehensive Benchmark for Evaluating the Robustness of LLMs as Mathematical Problem Solvers. ACL, 2024.

[7] F. Shi et al. Large Language Models Can Be Easily Distracted by Irrelevant Context. arXiv, 2023.

[8] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL, 2024.

[9] Z. Zhou et al. Can Language Models Perform Robust Reasoning in Chain-of-thought Prompting with Noisy Rationales? NeurIPS, 2024.

[10] D. Paul et al. Making Reasoning Matter: Measuring and Improving Faithfulness of Chain-of-Thought Reasoning. Findings of EMNLP, 2024.

[11] O. Press et al. Measuring and Narrowing the Compositionality Gap in Language Models. Findings of EMNLP, 2023.

[12] R. Jia and P. Liang. Adversarial Examples for Evaluating Reading Comprehension Systems. EMNLP, 2017.

[13] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.

[14] T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized Language Models. NeurIPS, 2023.

[15] J. Pereira et al. Targeted Adversarial Training for Natural Language Understanding. NAACL, 2021.